Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопсности
Высшая школа программной инженерии

ЛАБОРАТОРНАЯ РАБОТА №6

по дисциплине «Теория автоматов и формальных языков»

Выполнил
студент гр. 5130904/30108 Ребдев П.А.

Проверил Тышкевич А.И.

Санкт-Петербург
2026

1. Формулировка задания

1. Задать неавтоматный и не описываемый S-грамматикой язык с помощью синтаксических диаграмм.
2. Проверить пригодность диаграмм для разбора методом рекурсивного спуска. При невыполнении свойства внести изменения в исходные данные и обеспечить пригодность.
3. Построить МП-автомат отслеживания маршрута.
4. Разработать программную реализацию распознавателя по диаграммам (рекурсивный спуск).
5. Для выбранного слова (не менее 5 шагов вывода) привести протокол отслеживания маршрута МП-автоматом и протокол работы программного распознавателя (отладочная печать).

2. Исходный язык и его синтаксические диаграммы

В качестве основы взят язык арифметических выражений с операциями + и *, идентификаторами i и скобками (,). Язык не является автоматным (требуется подсчёт скобок и учёт приоритета операций) и не является S-грамматикой, поскольку содержит правила с пустой правой частью и правые части, начинающиеся с нетерминалов.

Грамматика G (продукции пронумерованы для ссылок):

№	Продукция
0	S → E $
1	E → T A
2	A → + T A
3	A → ε
4	T → F B
5	B → * F B
6	B → ε
7	F → i
8	F → (E)

Терминалы: +, *, i, (,), $

Нетерминалы: S, E, A, T, B, F

Начальный символ: S

2.1. Обозначения на диаграммах

Для наглядного представления синтаксических диаграмм используется нотация со следующими элементами:

- Начальный узел – start
- Конечный узел – stop
- Действие (action) – прямоугольник со скруглёнными углами, записывается в виде:действие;. Внутри может быть:
  - терминал (например, +, i) – обозначает необходимость сопоставить этот символ со входом;
  - имя нетерминала (например, E, T) – означает вызов соответствующей поддиаграммы.
- Ветвление (decision) – изображается с помощью if (условие) then (ветка) else (ветка) endif.
- Цикл (loop) – для реализации повторений используется while (условие) ... endwhile.
- Поток управления – стрелки, соединяющие узлы.

2.2. Синтаксические диаграммы

Диаграмма для S

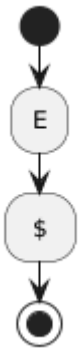


Диаграмма для E

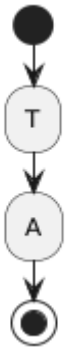
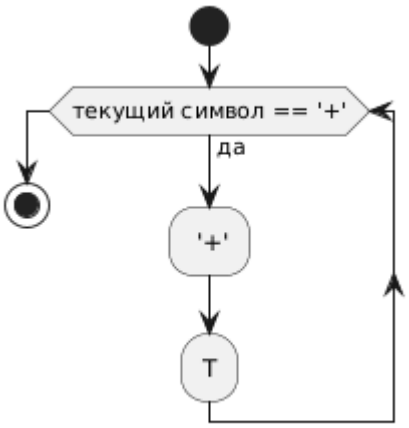


Диаграмма для A (повторение "+ T" ноль или более раз)



Пояснение: При входе в A проверяется текущий символ. Если это +, то выполняется последовательность: сопоставить +, вызвать T, и затем цикл повторяется. Если символ не + (т.е. принадлежит FOLLOW(A): $ или)), цикл завершается и происходит выход из диаграммы (ε-переход).

Диаграмма для T

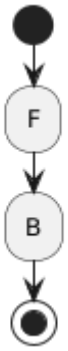
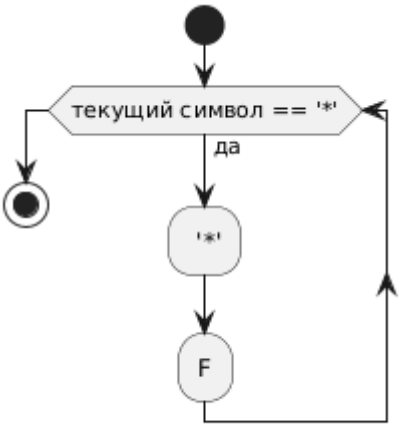
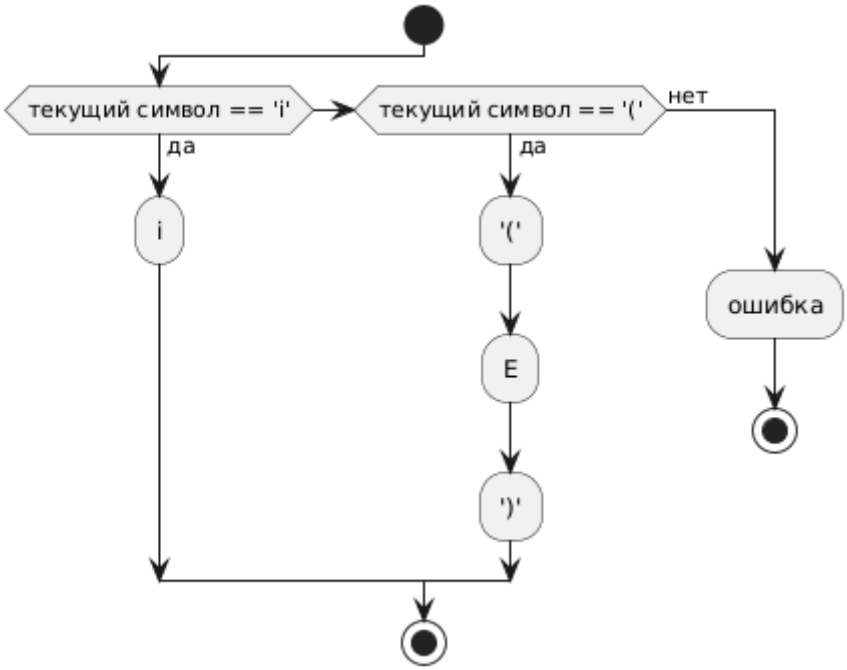


Диаграмма для B (повторение "* F" ноль или более раз)



Условие выхода: текущий символ ∈ FOLLOW(B) = +, $,).

Диаграмма для F (выбор между i и (E))



Пояснение: Проверяется текущий символ. Если i – выбирается путь с терминалом i. Если (– выбирается путь с открывающей скобкой, вызовом E и закрывающей скобкой. В противном случае – ошибка (на практике входная цепочка считается корректной, поэтому этот путь не должен достигаться при разборе успешных слов).

3. Проверка пригодности для рекурсивного спуска

Метод рекурсивного спуска требует, чтобы грамматика была LL(1): для каждого нетерминала множества LOOKAHEAD его альтернатив не пересекались. Выполним необходимые вычисления (аналогично лабораторной работе №5).

3.1. Вычисление FIRST

Нетерминал	FIRST
S	{ i , (}
E	{ i , (}
A	{ + }
T	{ i , (}
B	{ * }
F	{ i , (}

3.2. Вычисление FOLLOW

Нетерминал	FOLLOW
S	{ $ }
E	{ $,) }
A	{ $,) }
T	{ + , $,) }
B	{ + , $,) }
F	{ * , + , $,) }

3.3. Решающие множества LOOKAHEAD

Продукция	LOOKAHEAD
0: S → E $	{ i , (}
1: E → T A	{ i , (}
2: A → + T A	{ + }
3: A → ε	{ $,) }
4: T → F B	{ i , (}
5: B → * F B	{ * }
6: B → ε	{ + , $,) }
7: F → i	{ i }
8: F → (E)	{ (}

3.4. Проверка непересечения

- Для A: {+} ∩ {$,)} = ∅
- Для B: {*} ∩ {+,$,)} = ∅

- Для F: $\{i\} \cap \{(\} = \emptyset$

Все условия выполнены. Грамматика является LL(1), следовательно, пригодна для реализации рекурсивного спуска без возвратов.

4. МП-автомат отслеживания маршрута

Построим МП-автомат, моделирующий левосторонний вывод. Автомат имеет единственное состояние q. Магазин хранит символы грамматики (терминалы и нетерминалы), дно обозначается ⊥. Управление осуществляется по таблице, построенной из LOOKAHEAD-множеств.

4.1. Таблица переходов

	+	*	i	(	)	$
S			E$	E$		
E			TA	TA		
A	+TA				ε	ε
T			FB	FB		
B	ε	*FB			ε	ε
F			i	(E)		

4.2. Правила работы

- Если на вершине стека нетерминал X и текущий входной символ a, то по таблице выбирается продукция X → α. X заменяется на α (символы α заносятся в стек справа налево). Указатель входа не двигается.
- Если на вершине стека терминал x и он совпадает с текущим входным символом a, то оба удаляются (стек и вход) – указатель входа сдвигается.
- Если ни одно условие не выполнено – ошибка.

Начальная конфигурация: стек содержит S⊥, входная цепочка заканчивается символом $.

4.3. Отслеживание маршрута

При работе автомата каждая замена нетерминала на правую часть соответствует входу в диаграмму этого нетерминала и последующему движению по ней. Сопоставление терминала со входом соответствует прохождению терминального узла диаграммы. Когда на вершине стека оказывается нетерминал, мы как бы стоим в начале его диаграммы; замена на правую часть – это разворачивание диаграммы в последовательность узлов.

Ниже приведён протокол для слова i+i*i$ с подробными комментариями, указывающими, на какой диаграмме происходит движение и какие переходы выполняются.

№	Стек (вершина слева)	Остаток входа	Активная диаграмма (текущий нетерминал на вершине)	Комментарий
1	S ⊥	i + i * i $	S	Начало. Входим в диаграмму S.
2	E $ ⊥	i + i * i $	E	По таблице: S → E $. Замена S на E $. Теперь на вершине E – вход в диаграмму E.
3	T A $ ⊥	i + i * i $	T	E → T A. Раскрытие E. На вершине T – вход в диаграмму T.
4	F B A $ ⊥	i + i * i $	F	T → F B. Раскрытие T. На вершине F – вход в диаграмму F.
5	i B A $ ⊥	i + i * i $	F (терминал i)	F → i. Раскрытие F. Теперь на вершине терминал i.
6	B A $ ⊥	+ i * i $	B	Сопоставляем i со входом: удаляем i из стека и сдвигаем вход. Это означает, что мы прошли терминальный узел i на диаграмме F и возвращаемся в диаграмму T (теперь на вершине B – продолжение T).
7	A $ ⊥	+ i * i $	A	B → ε (текущий символ + ∈ FOLLOW(B)). Пустой переход: мы вышли из диаграммы B, вернулись в диаграмму E (на вершине A).
8	+ T A $ ⊥	+ i * i $	A (терминал +)	A → + T A. Раскрытие A. На вершине терминал + – узел диаграммы A.
9	T A $ ⊥	i * i $	T	Сопоставляем + со входом. Прошли терминал +, теперь на вершине T – вход в диаграмму T (второй раз).
10	F B A $ ⊥	i * i $	F	T → F B. Снова вход в F.
11	i B A $ ⊥	i * i $	F (терминал i)	F → i.
12	B A $ ⊥	* i $	B	Сопоставляем i. Возврат из F в T (на вершине B).
13	* F B A $ ⊥	* i $	B (терминал *)	B → * F B (текущий символ *). Раскрытие B, на вершине * – терминал.
14	F B A $ ⊥	i $	F	Сопоставляем *. Прошли *, на вершине F – снова вход в F.
15	i B A $ ⊥	i $	F (терминал i)	F → i.

16	B A $ ⊥	$	B	Сопоставляем i. Возврат в T (на вершине B).
17	A $ ⊥	$	A	B → ε (текущий символ $ ∈ FOLLOW(B)). Выход из B, теперь на вершине A.
18	$ ⊥	$	(терминал $)	A → ε ($ ∈ FOLLOW(A)). Выход из A, на вершине $ – терминал.
19	⊥			Сопоставляем $ со входом. Стек пуст, вход пуст – успех.

Примечание: Каждое появление нетерминала на вершине стека соответствует началу работы его диаграммы, а его замена – развёртыванию диаграммы. Удаление терминала – продвижение по диаграмме. Пустые замены (ε) соответствуют выходу из диаграммы без потребления входных символов.

5. Программная реализация распознавателя

Разработана программа на C++, реализующая рекурсивный спуск по синтаксическим диаграммам. Для каждого нетерминала написана функция, которая возвращает true, если из текущей позиции удаётся разобрать соответствующий фрагмент. Глобальная переменная pos хранит индекс текущего символа во входной строке. Включена отладочная печать, показывающая вход и выход из функций (отслеживание маршрута).

5.1. Исходный код

```
#include <iostream>
#include <string>

std::string input;
size_t pos;

void next()
{
  ++pos;
}

char current()
{
  if (pos < input.size()) return input[pos];
  return 0;
}

bool match(char expected)
{
  if (current() == expected)
  {
    next();
    return true;
  }
  return false;
}

void printIndent(int depth)
{
  for (int i = 0; i < depth; ++i) std::cout << "  ";
}

bool parseS(int depth);
bool parseE(int depth);
bool parseA(int depth);
bool parseT(int depth);
bool parseB(int depth);
bool parseF(int depth);

bool parseS(int depth)
{
  printIndent(depth); std::cout << "S: enter, current = " << current() << "\n";
  if (!parseE(depth + 1)) { printIndent(depth); std::cout << "S: fail\n"; return false; }
  if (!match('$')) { printIndent(depth); std::cout << "S: fail (expected $)\n"; return false; }
  printIndent(depth); std::cout << "S: success\n";
  return true;
}
```

```
bool parseE(int depth)
{
  printIndent(depth); std::cout << "E: enter, current = " << current() << "\n";
  if (!parseT(depth + 1)) { printIndent(depth); std::cout << "E: fail\n"; return false; }
  if (!parseA(depth + 1)) { printIndent(depth); std::cout << "E: fail\n"; return false; }
  printIndent(depth); std::cout << "E: success\n";
  return true;
}

bool parseA(int depth)
{
  printIndent(depth); std::cout << "A: enter, current = " << current() << "\n";
  if (current() == '+')
  {
    match('+');
    if (!parseT(depth + 1)) { printIndent(depth); std::cout << "A: fail\n"; return false; }
    if (!parseA(depth + 1)) { printIndent(depth); std::cout << "A: fail\n"; return false; }
    printIndent(depth); std::cout << "A: success (+\n";
    return true;
  }
  else if (current() == ')' || current() == '$')
  {
    printIndent(depth); std::cout << "A: success (epsilon)\n";
    return true;
  }
  printIndent(depth); std::cout << "A: fail\n";
  return false;
}

bool parseT(int depth)
{
  printIndent(depth); std::cout << "T: enter, current = " << current() << "\n";
  if (!parseF(depth + 1)) { printIndent(depth); std::cout << "T: fail\n"; return false; }
  if (!parseB(depth + 1)) { printIndent(depth); std::cout << "T: fail\n"; return false; }
  printIndent(depth); std::cout << "T: success\n";
  return true;
}

bool parseB(int depth)
{
  printIndent(depth); std::cout << "B: enter, current = " << current() << "\n";
  if (current() == '*')
  {
    match('*');
    if (!parseF(depth + 1)) { printIndent(depth); std::cout << "B: fail\n"; return false; }
    if (!parseB(depth + 1)) { printIndent(depth); std::cout << "B: fail\n"; return false; }
    printIndent(depth); std::cout << "B: success (*)\n";
    return true;
  }
  else if (current() == '+' || current() == ')' || current() == '$')
  {
    printIndent(depth); std::cout << "B: success (epsilon)\n";
    return true;
  }
  printIndent(depth); std::cout << "B: fail\n";
  return false;
}

bool parseF(int depth)
{
  printIndent(depth); std::cout << "F: enter, current = " << current() << "\n";
```

```
  if (current() == 'i')
  {
    match('i');
    printIndent(depth); std::cout << "F: success (i)\n";
    return true;
  }
  else if (current() == '(')
  {
    match('(');
    if (!parseE(depth + 1)) { printIndent(depth); std::cout << "F: fail\n"; return false; }
    if (!match(')')) { printIndent(depth); std::cout << "F: fail (expected ))\n"; return false; }
    printIndent(depth); std::cout << "F: success ((E))\n";
    return true;
  }
  printIndent(depth); std::cout << "F: fail\n";
  return false;
}

int main()
{
  std::cout << "Enter expression (end with $): ";
  std::getline(std::cin, input);
  pos = 0;
  if (parseS(0) && pos == input.size())
    std::cout << "ACCEPT\n";
  else
    std::cout << "REJECT\n";
  return 0;
}
```

5.2. Особенности реализации

- Функции parseX соответствуют нетерминалам.
- Выбор альтернативы осуществляется по текущему символу (current()), что соответствует LOOKAHEAD-множествам.
- Для ε-правил проверяется, допустим ли данный символ (согласно FOLLOW).
- Отладочная печать с отступами позволяет наглядно отследить маршрут рекурсивных вызовов.

6. Пример работы и протоколы распознавания

Для демонстрации возьмём слово i+i*i$ (длина вывода 12 шагов, что соответствует условию «не менее 5 шагов»).

6.1. Протокол работы МП-автомата

Протокол уже подробно разобран в п. 4.3. Здесь приведём его в компактной форме с указанием активной диаграммы в каждый момент.

№	Стек	Вход	Активная диаграмма (текущий нетерминал на вершине)
1	S	i+i*i$	S (начало)
2	E$	i+i*i$	E
3	TA$	i+i*i$	T
4	FBA$	i+i*i$	F
5	iBA$	i+i*i$	F (терминал i)
6	BA$	+i*i$	B (после сопоставления i)
7	A$	+i*i$	A (после ε-выхода из B)
8	+TA$	+i*i$	A (терминал +)
9	TA$	i*i$	T (после сопоставления +)
10	FBA$	i*i$	F
11	iBA$	i*i$	F (терминал i)
12	BA$	*i$	B
13	*FBA$	*i$	B (терминал *)
14	FBA$	i$	F
15	iBA$	i$	F (терминал i)
16	BA$	$	B
17	A$	$	A (после ε-выхода из B)
18	$	$	(терминал $)
19			успех

Примечание: Каждый раз, когда на вершине появляется нетерминал, мы входим в его диаграмму; когда нетерминал заменяется правой частью, мы движемся по ней; сопоставление терминала – прохождение листа; ε-переход – немедленный выход из диаграммы.

6.2. Протокол работы рекурсивного спуска

При запуске программы с входом i+i*i$ отладочная печать даёт следующий вывод (глубина отступов соответствует вложенности вызовов):

```
Enter expression (end with $): S: enter, current = i
  E: enter, current = i
    T: enter, current = i
      F: enter, current = i
      F: success (i)
      B: enter, current = +
      B: success (epsilon)
    T: success
    A: enter, current = +
      T: enter, current = i
        F: enter, current = i
        F: success (i)
        B: enter, current = *
          F: enter, current = i
          F: success (i)
          B: enter, current = $
          B: success (epsilon)
        B: success (*)
      T: success
      A: enter, current = $
      A: success (epsilon)
    A: success (+
  E: success
S: success
ACCEPT
```

Каждый вызов сопровождается сообщением о входе и выходе. Видно, что выбор альтернатив осуществляется строго по текущему символу и маршрут полностью детерминирован.

6.3. Сравнение протоколов

Оба протокола отражают один и тот же процесс нисходящего разбора:

МП-автомат показывает изменения стека и входной ленты.

Рекурсивный спуск показывает иерархию вызовов функций, соответствующую дереву разбора.

Последовательность применённых правил совпадает с полученной в лабораторной работе №5:
(1) S -> E$
(2) E -> TA
(3) T -> FB
(4) F -> i
(5) B -> ε
(6) A -> +TA
(7) T -> FB
(8) F -> i
(9) B -> *FB
(10) F -> i
(11) B -> ε
(12) A -> ε

7. Выводы

1. Для неавтоматного языка арифметических выражений построены синтаксические диаграммы в виде UML-диаграмм активностей, наглядно представляющие грамматику.
2. Выполнена проверка LL(1)-свойства, подтвердившая пригодность грамматики для разбора методом рекурсивного спуска.
3. Построен МП-автомат, отслеживающий маршрут распознавания, и приведена его таблица переходов.
4. Реализована программа на C++, осуществляющая рекурсивный спуск в соответствии с диаграммами; программа включает детальную отладочную печать.
5. Для слова i+i*i$ получены и сопоставлены протоколы работы МП-автомата и рекурсивного спуска; оба метода дают идентичную последовательность применённых правил и успешно распознают слово.
6. Разработанные средства могут быть использованы для построения синтаксических анализаторов реальных языков программирования.